CI / CD Pipeline

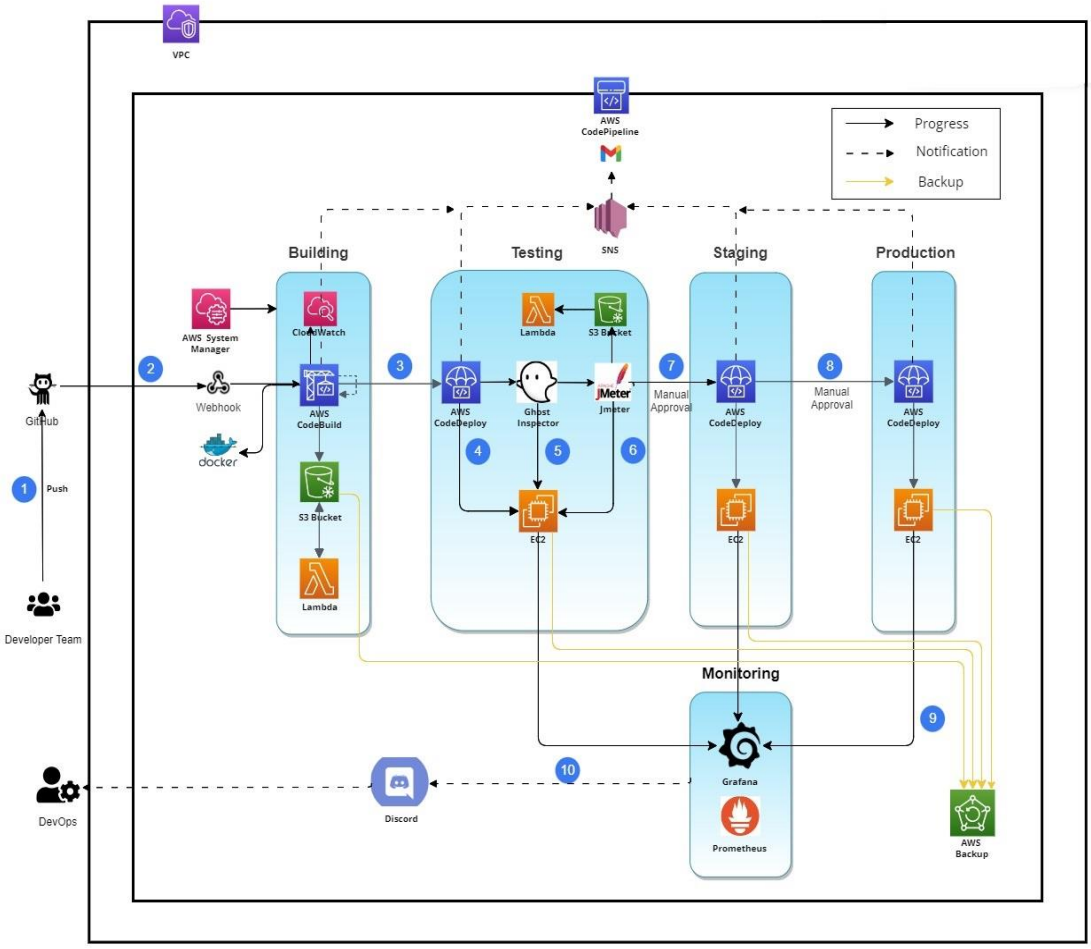

Fig.1 CI/CD workflow

Workflow Explanation

1. **Pull Request**: - Firstly, developer will do pull request to main git repository to implement any changes to main branch.
   1.1 For code hosting and version control we use **GitHub.**
   1.2 Whenever changes are made it will trigger pull requests.
   1.3 Before adding changes into main branch it will be verified by the developer team head and he will approve it for build.

2. **Code Pipeline: -** It will trigger the Code Pipeline, which is connected with GitHub through **webhook**.

   2.1 **Source:-** First stage of code pipeline is to take changes from source (i.e. GitHub)

   2.2 **Code Build:-** Second stage is **Code Build** where the building process of code takes place.
      2.2.1 After starting building it will send the notification of build status to developer through mail.
      2.2.2 The required dependencies will be installed according to code written in buildspec.yml file and the code will be tested in that environment.
      2.2.3 Docker image will be created of the new version and image will be push to docker repo.
      2.2.4 Login details of docker will be stored in **Amazon System manager** and can be accessed through parameter-store in code build.
      2.2.5 The build result will be sent to developers through **SNS service** in mail.
      2.2.6 The artifacts will be stored in **S3 bucket** and if new file will be added then it will trigger lambda function to delete old version artifacts.
      2.2.7 Backup of S3 bucket (artifacts) will taken by **AWS backup** service and backup can be restored from recovery point arn of backup.
      2.2.8 The logs of code build are stored in **CloudWatch** events.

2.3 **Code deployment**: - After successful building, the deployment will take on **EC2** instance using **Code Deploy** service.

2.3.1 Code deployment has lifecycle event consist of 7 events which is run in order.

2.3.2 The scripts for deployment is written in appspec.yml file and this file is uploaded to **GitHub**.

2.3.3 The code deployment start from the build artifacts which are stored in s3 bucket uploaded from code build.

2.3.4 The codedeploy-agent is being installed in EC2 instance for deployment process and make sure that codedeploy-agent status is running in instance.

2.3.5 Application Stop is first event which is run to delete old version container running on instance and to delete old version image which is not being used.

2.3.6 After Install event has script for pulling of the latest image push in **Docker** and run that image on 3001 port of instance.

2.3.7 The status of deployment is sent to developers through **SNS service** in mail.

2.4 **UI-Testing: -** After successfully deployment the site will be live on 3001 port of instance and for UI-testing we have configured **Ghost Inspector.**

2.4.1 The test suite are already configured in ghost inspector website using extension.

2.4.2 The live site will be test according to test suites and after successfully passing test it will generate video of test.

2.5 **Load Testing:-** The load testing is carried out in code build service where all scripts are written in buildspec.yml file.

2.5.1 Firstly, when the instance will be allocated for building, **JMeter** will be installed in that environment.

2.5.2 The jmeter configuration is written in file named react-app.jmx which is already uploaded to GitHub.

2.5.3 The file is run using Jmeter cli command and the result will be stored in .csv file , the dashboard generated for visualisation of result will be stored as zip.

2.5.4 Both zip and csv file are uploaded to S3 bucket using AWS CLI.

.

2.5.5 The access key and secret key for AWS CLI are stored in **AWS System manager** and can be accessed through parameter-store.

2.5.6 After uploading files, **lambda** function is triggered to delete old zip and csv files.

2.6 **Performance Testing: -** This test is carried out by **Grafana.**

2.6.1 The performance testing is carried out for EC2 instance, so Grafana is being installed in instance for monitoring.

2.6.2 Grafana installed will be run on 3000 port of instance and it will required metrics to generate graph.

2.6.3 **Prometheus** is installed for collecting metrics of instance and it runs on port 9090.

2.6.4 Grafana is connected with Prometheus to convert metrics data into graphical form.

2.6.5 CPU utilization, memory usage, disk read and write throughput are some graph which are showed in dashboard.

2.6.6 The alarm is set for high threshold value of CPU utilization and if it hits the maximum threshold value then it fires alarm, notifies developer through **Discord** which is connected to Grafana by webhook.

3. **Staging:-** This stage requires manual approval of developer team head.

3.1 A staging environment is a copy of live website and is the last step in the deployment process before changes are deployed to the live website.

3.2 The performance of staging instance is monitor using Grafana.

4. **Production:-** After successful staging , the website is deployed in production environment.

4.1 The performance of production instance is also monitor through Grafana.

Components

- **AWS CodePipeline**
  - AWS CodePipeline is a continuous delivery service that enables you to model, visualize, and automate the steps required to release your software.

- **GitHub**
  - GitHub is an Internet hosting service for software development and version control using Git.
  - It provides the distributed version control of Git plus access control, bug tracking, software feature requests, task management, continuous integration for every project.

- **AWS CodeBuild**
  - AWS CodeBuild is a fully managed continuous integration service that compiles source code, runs tests, and produces ready-to-deploy software packages.
  - It provides prepackaged build environments for popular programming languages and build tools such as Apache Maven, Gradle, and more.

- **AWS S3**
  - Amazon Simple Storage Service (Amazon S3) is an object storage service offering industry-leading scalability, data availability, security, and performance.
  - Amazon S3 used to store and retrieve any amount of data at any time, from anywhere.

- **AWS EC2**
  - Amazon Elastic Compute Cloud (Amazon EC2) is a web service that provides secure, resizable compute capacity in the cloud. It is designed to make web-scale cloud computing easier for developers.

- **AWS CodeDeploy**
  - CodeDeploy is a deployment service that automates application deployments to Amazon EC2 instances, on-premises instances, serverless Lambda functions, or Amazon ECS services.

- **Ghost Inspector**
  - Ghost Inspector is an automated browser testing and monitoring service that checks for problems with your website or application.
  - It carries out operations in a browser, the same way a user would, to ensure that everything is working properly.

- **Grafana**
  - Grafana is an open-source data visualization and monitoring tool that allows users to visualize, analyze and understand data from various sources.
  - It provides a flexible and powerful platform for creating and sharing dashboards.

- **AWS Secret Manager**
  - AWS Secret Manager is a secrets management service that helps you protect access to your applications, services, and IT resources.
  - This service enables you to easily rotate, manage, and retrieve database credentials, API keys, and other secrets throughout their lifecycle.

- **AWS Backup**
  - AWS Backup is a fully-managed service that makes it easy to centralize and automate data protection across AWS services, in the cloud, and on premises.
  - Using this service, you can configure backup policies and monitor activity for your AWS resources in one place.

- **CloudWatch**
  - Amazon CloudWatch monitors your Amazon Web Services (AWS) resources and the applications you run on AWS in real time.
  - CloudWatch is use to collect and track metrics, which are variables that can be measured for our resources and applications.

- **Docker**
  - Docker is a set of platforms as a service (PaaS) products that use Operating system-level virtualization to deliver software in packages called containers.

- Containers are isolated from one another and bundle their own software, libraries, and configuration files; they can communicate with each other through well-defined channels.
- All containers are run by a single operating system kernel and therefore use fewer resources than a virtual machine.

- **Jmeter**
  - You can use JMeter to analyze and measure the performance of web application or a variety of services.
  - JMeter originally is used for testing Web Application or FTP application.